JavaScript po ES6

ES2016 až ES2024

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

02.10.2026

CC BY-NC-SA 4.0

Úvod

Každoroční vydání

Minule jsme probrali **ES6 (ES2015)** – revoluci JavaScriptu

Od té doby vychází každý rok **nová verze** standardu ECMAScript

Změny jsou menší, ale **praktické** – dnes si projdeme verze **ES2016 až ES2024**

Čekají nás mimo jiné:

- `async`/`await`
- optional chaining `?.` a nullish coalescing `??`
- private fields, top-level await a immutable metody polí

ES7/ES2016

ES7 (ES2016) přinesl dvě hlavní novinky:

1. Operátor exponenciace (**)
2. Array.prototype.includes()

Operátor exponenciace (**)

Operátor exponenciace (**) je nový způsob zápisu umocňování

```
// Old way
console.log(Math.pow(2, 3));  // 8

// New way
console.log(2 ** 3);  // 8
```

Operátor exponenciace (**)

Funguje i s přiřazením:

```
let num = 2;
num **= 3;  // num = num ** 3
console.log(num);  // 8
```

Array.prototype.includes

includes je nová metoda pro kontrolu existence prvku v poli

```
const numbers = [1, 2, 3, 4, 5];

// Old way
console.log(numbers.indexOf(3) !== -1);  // true
console.log(numbers.indexOf(6) !== -1);  // false

// New way
console.log(numbers.includes(3));  // true
console.log(numbers.includes(6));  // false
```

Array.prototype.includes

includes má lepší podporu pro `NaN` a funguje s fromIndex parametrem

```
// Problem with NaN in indexOf
const arr = [1, NaN, 2, 3];
console.log(arr.indexOf(NaN));     // -1 (not found)
console.log(arr.includes(NaN));    // true (found)
```

Array.prototype.includes

```
// Using fromIndex
const numbers = [1, 2, 3, 1, 2, 3];

console.log(numbers.includes(2, 3));
// true (searches from index 3)

console.log(numbers.includes(2, -2));
// true (searches last 2 elements)
```

ES8/ES2017

ES8 (ES2017) přinesl několik významných novinek:

1. async/await
2. Object.entries() a Object.values()
3. String padding (padStart, padEnd)
4. Trailing commas v parametrech funkcí
5. Object.getOwnPropertyDescriptors()

Async/Await

async/await je nový způsob práce s Promises, který dělá asynchronní kód čitelnější

Místo řetězení `.then()` lze použít `await` přímo v kódu

Async/Await

```
// Promise way
function fetchUserData() {
  return fetch('/api/user')
    .then(response => response.json())
    .then(user => {
      return fetch(`/api/posts/${user.id}`);
    })
    .then(response => response.json());
}
```

Async/Await

```
// async/await way
async function fetchUserData() {
  const userResponse = await fetch('/api/user');
  const user = await userResponse.json();
  const postsResponse = await fetch(`/api/posts/${user.id}`);
  return postsResponse.json();
}
```

Async/Await

Error handling s async/await:

```
// Promise way
function fetchData() {
  return fetch('/api/data')
    .then(response => response.json())
    .catch(error => {
      console.error('Error:', error);
      throw error;
    });
}
```

Async/Await

```
// async/await way
async function fetchData() {
  try {
    const response = await fetch('/api/data');
    return await response.json();
  } catch (error) {
    console.error('Error:', error);
    throw error;
  }
}
```

Async/Await

Paralelní operace s async/await:

```
// Sequential processing
async function fetchSequential() {
  const user = await fetchUser();
  const posts = await fetchPosts();
  const comments = await fetchComments();
  return { user, posts, comments };
}
```

Async/Await

```
// Parallel processing
async function fetchParallel() {
  const [user, posts, comments] = await Promise.all([
    fetchUser(),
    fetchPosts(),
    fetchComments()
  ]);
  return { user, posts, comments };
}
```

Object.keys

Již v ES5 byla k dispozici metoda `Object.keys`

```
const person = {
  name: 'John',
  age: 30,
  city: 'New York'
};
console.log(Object.keys(person));
// ['name', 'age', 'city']
```

Object.values

Nová metoda **Object.values()** vrací pole hodnot objektu

```
const person = {
  name: 'John',
  age: 30,
  city: 'New York'
};

console.log(Object.values(person));
// ['John', 30, 'New York']
```

Object.values

```
// Practical example
const total = Object.values(person)
  .filter(value => typeof value === 'number')
  .reduce((sum, num) => sum + num, 0);
```

Object.entries

Další nová metoda **Object.entries()** vrací pole dvojic [klíč, hodnota]

```
const person = {
  name: 'John',
  age: 30,
  city: 'New York'
};
```

Object.entries

```
for (const [key, value] of Object.entries(person)) {
  console.log(`${key}: ${value}`);
}
// name: John
// age: 30
// city: New York

// Convert to Map
const map = new Map(Object.entries(person));
```

String padding

padStart a **padEnd** pro zarovnání řetězců

```
// padStart - adds characters to the beginning
console.log('5'.padStart(2, '0'));     // "05"
console.log('123'.padStart(5, '0'));   // "00123"
console.log('5'.padStart(4, '*'));     // "***5"

// padEnd - adds characters to the end
console.log('5'.padEnd(2, '0'));       // "50"
console.log('123'.padEnd(5, '0'));     // "12300"
console.log('5'.padEnd(4, '*'));       // "5***"
```

String padding

Praktické použití string paddingu:

```
// Number formatting
const numbers = [1, 23, 456];
numbers.forEach(num => {
  console.log(num.toString().padStart(3, '0'));
});
// "001"
// "023"
// "456"
```

String padding

```
// Text padding
const items = ['Apple', 'Banana', 'Orange'];
const maxLength = Math.max(
  ...items.map(item => item.length)
);
items.forEach(item => {
  console.log(item.padEnd(maxLength, ' ') + '|');
});
// "Apple  |"
// "Banana |"
// "Orange |"
```

ES9/ES2018

ES9 (ES2018) přinesl několik důležitých vylepšení:

1. Rest/Spread vlastnosti pro objekty
2. Promise.prototype.finally()
3. Asynchronní iterace (for-await-of)
4. Vylepšení regulárních výrazů

Rest/Spread pro objekty

Rest a **spread** operátory nyní fungují i s objekty

```
// Rest pro objekty
const { name, age, ...rest } = {
  name: 'John',
  age: 30,
  city: 'New York',
  country: 'USA'
};
console.log(name); // 'John'
console.log(age);  // 30
console.log(rest); // { city: 'New York', country: 'USA' }
```

Rest/Spread pro objekty

Spread pro objekty:

```
const person = {
  name: 'John',
  age: 30
};

// Copying object
const clone = { ...person };
```

Rest/Spread pro objekty

```
// Extending object
const extended = {
  ...person,
  city: 'New York',
  age: 31  // overrides original value
};

console.log(extended);
// { name: 'John', age: 31, city: 'New York' }
```

Promise.prototype.finally

finally se volá vždy po dokončení Promise, bez ohledu na výsledek

Promise.prototype.finally

```
function fetchData() {
  showLoadingSpinner();

  return fetch('/api/data')
    .then(response => response.json())
    .catch(error => {
      console.error('Error:', error);
      throw error;
    })
    .finally(() => { hideLoadingSpinner(); });
}
```

Asynchronní iterace

for-await-of umožňuje iterovat přes asynchronní iterátory

```
async function* asyncGenerator() {
  yield Promise.resolve(1);
  yield Promise.resolve(2);
  yield Promise.resolve(3);
}
```

Asynchronní iterace

```
for await (const num of asyncGenerator()) {
  console.log(num);
}
// 1
// 2
// 3
```

Asynchronní iterace

Praktické použití s fetch:

```
async function* fetchCommits(repo) {
  let url = `https://api.github.com/repos/${repo}/commits`;
  while (url) {
    const response = await fetch(url);
    const data = await response.json();
    for (let commit of data) { yield commit; }
    // Reads the link header to get the next page
    url = getNextPage(response.headers);
  }
}
```

Asynchronní iterace

```
async function showCommits(repo) {
  try {
    for await (const commit of fetchCommits(repo)) {
      console.log(commit.sha);
    }
  } catch (err) {
    console.error(err);
  }
}
```

ES10/ES2019

ES10 (ES2019) přinesl několik užitečných vylepšení:

1. Array.prototype.flat() a flatMap()
2. Object.fromEntries()
3. String.prototype.trimStart() a trimEnd()
4. Optional catch binding
5. Stabilní Array.sort()

Array.prototype.flat a flatMap

flat zploští vnořená pole do jednoho pole

```
const arr = [1, 2, [3, 4, [5, 6]]];

console.log(arr.flat());     // [1, 2, 3, 4, [5, 6]]
console.log(arr.flat(2));    // [1, 2, 3, 4, 5, 6]
console.log(arr.flat(Infinity)); // [1, 2, 3, 4, 5, 6]

// Removes empty items
const sparse = [1, 2, , 4, 5];
console.log(sparse.flat()); // [1, 2, 4, 5]
```

Array.prototype.flat a flatMap

flatMap kombinuje map a flat do jedné operace

```
// Without flatMap
const sentences = ["Hello world", "JavaScript is fun"];
const words = sentences
  .map(s => s.split(" "))
  .flat();

// With flatMap
const words2 = sentences.flatMap(s => s.split(" "));
console.log(words2); // ["Hello", "world", "JavaScript",
"is", "fun"]
```

```
// Practical example
const numbers = [1, 2, 3];
console.log(numbers.flatMap(x => [x, x * 2]));
// [1, 2, 2, 4, 3, 6]
```

Object.fromEntries

Object.fromEntries je opak Object.entries - převádí pole dvojic na objekt

```
const entries = [
  ['name', 'John'],
  ['age', 30],
  ['city', 'New York']
];
const obj = Object.fromEntries(entries);
console.log(obj);
// { name: "John", age: 30, city: "New York" }
```

Object.fromEntries

```
// Filtering object
const person = {
  name: 'John', age: 30,
  password: '123456'
};

const filtered = Object.fromEntries(
  Object.entries(person)
    .filter(([key]) => !key.includes('password'))
);
console.log(filtered); // { name: "John", age: 30 }
```

Object.fromEntries

```
// Transforming values
const prices = {
  banana: 1, apple: 2, orange: 3
};

const doubled = Object.fromEntries(
  Object.entries(prices)
    .map(([key, value]) => [key, value * 2])
);
console.log(doubled);
// { banana: 2, apple: 4, orange: 6 }
```

String.prototype.trimStart a trimEnd

trimStart a **trimEnd** odstraňují bílé znaky ze začátku/konce řetězce

```
const str = '   Hello world   ';

// Old methods
console.log(str.trimLeft());  // "Hello world   "
console.log(str.trimRight()); // "   Hello world"
// New methods
console.log(str.trimStart()); // "Hello world   "
console.log(str.trimEnd());   // "   Hello world"
// Combination
console.log(str.trimStart().trimEnd()); // "Hello world"
```

ES11/ES2020

ES11 (ES2020) přinesl několik významných novinek:

1. BigInt
2. Nullish coalescing operátor (??)
3. Optional chaining (?.)
4. Promise.allSettled
5. globalThis
6. Dynamic import

BigInt

BigInt je nový datový typ pro práci s velkými celými čísly

```
// Creating BigInt
const bigInt = 9007199254740991n;  // n suffix
const alsoHuge = BigInt(9007199254740991);  // function

// Operations with BigInt
console.log(bigInt + 1n); // 9007199254740992n
console.log(bigInt * 2n); // 18014398509481982n
```

BigInt

```
// Comparison
console.log(1n < 2n);  // true
console.log(2n > 1n);  // true
console.log(2n === 2); // false
console.log(2n == 2);  // true
```

Nullish coalescing operátor (??)

Nullish coalescing operátor (??) poskytuje způsob, jak pracovat s `null` nebo `undefined`

```
// Problem with || operator
const count = 0;
const result = count || 42;  // 42

// Solution with ?? operator
const count = 0;
const result = count ?? 42;  // 0
```

Nullish coalescing operátor (??)

```
const name = null;
const text = name ?? "anonymous";  // "anonymous"
```

Nullish coalescing operátor (??)

Rozdíl mezi ?? a ||:

```
// || returns first "truthy" value
console.log(false || 'default');     // "default"
console.log(0 || 'default');         // "default"
console.log('' || 'default');        // "default"
console.log(null || 'default');      // "default"
console.log(undefined || 'default'); // "default"
```

Nullish coalescing operátor (??)

```
// ?? returns first value that is not null/undefined
console.log(false ?? 'default');     // false
console.log(0 ?? 'default');         // 0
console.log('' ?? 'default');        // ""
console.log(null ?? 'default');      // "default"
console.log(undefined ?? 'default'); // "default"
```

Optional chaining (?.)

Optional chaining (?.) umožňuje bezpečný přístup k vnořeným vlastnostem

```
const user = {
  name: 'John',
  address: {
    street: 'Main St'
  }
};
```

Optional chaining (?.)

```
// Old way (junior level developer)
const zipCode = user ?
                user.address ?
                user.address.zipCode : null : null;

// Old way
const zipCode = user && user.address && user.address.zipCode;

// New way
const zipCode = user?.address?.zipCode;
```

Optional chaining (?.)

Optional chaining s metodami a poli:

```
// Methods
const result = obj.method?.();
// undefined if method doesn't exist

// Array
const item = arr?.[0];
// undefined if arr is null/undefined
```

Optional chaining (?.)

```
// Combination
const zip = user?.address?.zip?.toString();

// Practical example
const userRole =
  user?.permissions?.role?.toUpperCase() ?? 'GUEST';
```

Promise.allSettled

Promise.allSettled čeká na dokončení všech promises, bez ohledu na výsledek - můžou selhat

```
const promises = [
  fetch('/api/user'),
  fetch('/api/posts'),
  fetch('/api/comments')
];
```

Promise.allSettled

```
// Promise.all fails on first error
Promise.all(promises)
  .then(results =>
    console.log('All succeeded:', results)
  )
  .catch(error =>
    console.log('At least one failed:', error)
  );
```

Promise.allSettled

```
// Promise.allSettled waits for all
Promise.allSettled(promises)
  .then(results => {
    results.forEach(result => {
      if (result.status === 'fulfilled') {
        console.log('Success:', result.value);
      } else {
        console.log('Error:', result.reason);
      }
    });
  });
```

ES12/ES2021

ES12 (ES2021) přinesl několik užitečných vylepšení:

1. Logical assignment operátory (&&=, ||=, ??=)
2. Numeric separators
3. String.prototype.replaceAll
4. Promise.any
5. WeakRef a FinalizationRegistry

Logical assignment operátory

Logical assignment operátory kombinují logické operátory s přiřazením

```
// OR assignment (||=)
let x = null;
x ||= 42;  // x = x || 42
console.log(x);  // 42
```

Logical assignment operátory

```
// AND assignment (&&=)
let y = 1;
y &&= 2;  // y = y && 2
console.log(y);  // 2

// Nullish assignment (??=)
let z = null;
z ??= 42;  // z = z ?? 42
console.log(z);  // 42
```

Logical assignment operátory

Praktické použití logical assignment operátorů:

```
const user = {
  name: 'John',
  settings: {
    theme: 'light'
  }
};
```

Logical assignment operátory

```
// Setting default values
user.settings.notifications ??= true;

// Conditional updates
user.isAdmin &&= user.permissions.includes('admin');

// Preserve existing value
user.role ||= 'user';
```

Numeric separators

Numeric separators (_) zlepšují čitelnost čísel

```
// Large numbers
const billion = 1_000_000_000;
const budget = 1_234_567.89;
// Binary numbers
const binary = 0b1010_0001_1000_0101;
// Hexadecimal numbers
const hex = 0xFF_EC_DE_5E;
// BigInt
const bigNumber = 1_234_567_890_123_456_789n;
```

String.prototype.replaceAll

replaceAll nahradí všechny výskyty podřetězce

```
// Old way
const text = 'hello hello hello';
console.log(text.replace(/hello/g, 'hi')); // 'hi hi hi'

// New way
console.log(text.replaceAll('hello', 'hi')); // 'hi hi hi'
```

Promise.any

Promise.any vrátí první úspěšně dokončený Promise

```
const promises = [
  fetch('https://api1.example.com/data'),
  fetch('https://api2.example.com/data'),
  fetch('https://api3.example.com/data')
];
```

Promise.any

```
// Returns first successful result
Promise.any(promises)
  .then(firstSuccess => {
    console.log('First success:', firstSuccess);
  })
  .catch(error => {
    // AggregateError if all fail
    console.log('All promises failed:', error);
  });
```

Promise.race a Promise.any

```
// Promise.race returns first completed (success or error)
Promise.race([
  fetch('/endpoint-1'),
  fetch('/endpoint-2')
]).then(/* first completed */);

// Promise.any returns first successful
Promise.any([
  fetch('/endpoint-1'),
  fetch('/endpoint-2')
]).then(/* first successful */);
```

Promise.race a Promise.any

```
// Practical example - fallback API
Promise.any([
  fetch('https://api.example.com'),
  fetch('https://api.secondary.example.com')
]).then(response => response.json());
```

ES13/ES2022

ES13 (ES2022) přinesl několik významných vylepšení:

1. Top-level await
2. Array.prototype.at()
3. Object.hasOwn()
4. Class fields a private methods
5. Static initialization blocks

Top-level await

Top-level await umožňuje použít await **mimo async funkce**

```
// Before - await only in async functions
async function getData() {
  const response = await fetch('/api/data');
  return response.json();
}

// Now - await directly in module
const response = await fetch('/api/data');
const data = await response.json();
export { data };
```

Top-level await

Praktické použití top-level await:

```
// config.js
const config = await fetch('/api/config');
export default await config.json();

// database.js
const connection = await createConnection();
export { connection };
```

Top-level await

```
// app.js
import { data } from './data.js';
// Module loads after await completes
console.log(data);
```

Array.prototype.at()

at() poskytuje lepší způsob přístupu k prvkům pole pomocí indexu

```
const arr = [1, 2, 3, 4, 5];

// Old way
console.log(arr[arr.length - 1]); // 5
console.log(arr[arr.length - 2]); // 4

// New way
console.log(arr.at(-1)); // 5
console.log(arr.at(-2)); // 4
console.log(arr.at(1));  // 2
```

Class fields a private methods

Private fields a **methods** přinášejí skutečné soukromé vlastnosti do tříd

Class fields a private methods

```
class Counter {
  #count = 0;      // private field
  #increment() {  // private method
    this.#count++;
  }

  getCount() { return this.#count; }

  increment() { this.#increment(); }
}
```

Class fields a private methods

```
const counter = new Counter();

counter.increment();
console.log(counter.getCount()); // 1
console.log(counter.#count); // SyntaxError
```

Singleton pomocí private fields

Statické private fields a methods:

```
class Singleton {
  static #instance;

  constructor() {
    if (Singleton.#instance) {
      throw new Error(
        'Use Singleton.getInstance() instead of new.'
      );
    }
  }
```

Singleton pomocí private fields

```
  static getInstance() {
    if (!Singleton.#instance) {
      Singleton.#instance = new Singleton();
    }
    return Singleton.#instance;
  }
}

const instance = Singleton.getInstance();
const instance2 = Singleton.getInstance();
console.log(instance === instance2); // true
```

ES14/ES2023 a ES15/ES2024

Nejnovější verze ES14 (ES2023) a ES15 (ES2024) přinesly:

ES14/ES2023:

- Array.prototype.toSorted, toReversed, with
- Array.prototype.findLast a findLastIndex
- Shebang podpora

ES15/ES2024:

- Object.groupBy a Map.groupBy
- Promise.withResolvers

Immutable metody polí

Immutable metody polí nemodifikují původní pole, ale vrací nová

```
const numbers = [1, 2, 3, 4];

// Old methods modify original array
numbers.reverse();
console.log(numbers); // [4, 3, 2, 1]
// New methods create new array
const original = [1, 2, 3, 4];
const reversed = original.toReversed();
console.log(original); // [1, 2, 3, 4]
console.log(reversed); // [4, 3, 2, 1]
```

Immutable metody polí

```
// Other new methods
console.log(original.toSorted(
  (a, b) => b - a)
); // [4, 3, 2, 1]

console.log(original.with(1, 10)); // [1, 10, 3, 4]
```

Seskupovací metody

Object.groupBy a Map.groupBy:

```
const items = [
  { type: "fruit", name: "apple" },
  { type: "vegetable", name: "carrot" },
  { type: "fruit", name: "banana" }
];
```

Seskupovací metody

```
// Object.groupBy
const grouped = Object.groupBy(items, item => item.type);
// {
//   fruit: [
//     { type: "fruit", name: "apple" },
//     { type: "fruit", name: "banana" }
//   ],
//   vegetable: [
//     { type: "vegetable", name: "carrot" }
//   ]
// }
```

Seskupovací metody

```
// Map.groupBy
const groupedMap = Map.groupBy(items, item => item.type);
// Map {
//   fruit: [
//     { type: "fruit", name: "apple" },
//     { type: "fruit", name: "banana" }
//   ],
//   vegetable: [
//     { type: "vegetable", name: "carrot" }
//   ]
// }
```

Shrnutí

Vývoj JavaScriptu

1. Lepší syntax a čitelnost kódu
   - let/const, arrow funkce, template literals
2. Moderní funkce pro práci s daty
   - destructuring, spread/rest, Map/Set
3. Vylepšené OOP
   - třídy, private fields, static blocks
4. Asynchronní programování
   - Promises, async/await, top-level await
5. Nové operátory a metody
   - ??, ?., ||=, ??=, at(), groupBy()

Vývoj JavaScriptu

Doporučení pro moderní JavaScript:

1. Používejte nové funkce pro čitelnější kód
2. Věnujte pozornost kompatibilitě prohlížečů
3. Využívejte transpilery (Babel) pro starší prohlížeče
4. Sledujte nové verze a vylepšení
5. Používejte TypeScript pro lepší typovou bezpečnost